



CMPT 295

Unit - Machine-Level Programming

Lecture 18 – Assembly language –

Function Call and Stack - Passing Data – Part 1

Last Lecture

- **Function call** mechanisms in **x86-64**:
 - 1) Mechanisms for **passing control**
- **x86-64 stack** register and instructions:
 - Register stack pointer: **%rsp**,
 - Instructions: **pushq** and **popq**
- **x86-64** function call instructions:
 - **call** and **ret**

Today's Menu

- Introduction
 - C program -> assembly code -> machine level
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – **Stack** – Recursion
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point operations

Overview of Function Call

What happens when a function (**caller**) calls another function (**callee**) in **x86-64 Assembly Language**?

1. **Control** (execution flow)
2. **Data**
3. **Memory** -> stack memory segment

Looking into the **function call conventions** of **x86-64 Assembly Language**

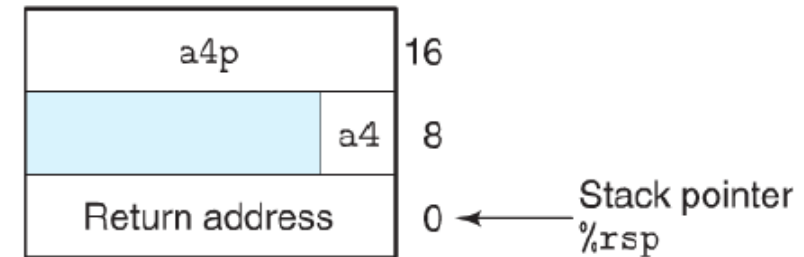
8

A note about the *stack diagrams* in our textbook!

- Sometimes textbook displays `%rsp` on the right of the *stack diagram*:

Figure 3.30

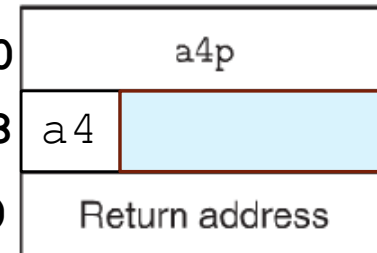
Stack frame structure for function `proc`. Arguments `a4` and `a4p` are passed on the stack.



- In our lecture notes (and other resources), we draw the `%rsp` on the left of the *stack diagram*:

Here, we have reformatted the above *stack diagram* into the format we use in our lecture notes:

$16_{10} \rightarrow 10_{16} \rightarrow 0x10$
 $8_{10} \rightarrow 8_{16} \rightarrow 0x8$
Stack pointer `%rsp` → $0x0$



2. *Passing data* mechanism – using **stack** **x86-64** function call convention

1. **Caller** and **callee** functions must obey **function call convention** (protocol) when passing data during function call

➤ **Caller:**

- Before calling the **callee** function, the **caller** must ensure that the **callee's arguments** (1 to 6) are loaded into their specific registers:

If there is a ... **1st argument**, then it is moved to **%rdi** (or **%edi**, or **%di** or **%dil**)

If there is a ... **2nd argument**, then it is moved to **%rsi** (or **%esi**, or **%si** or **%sil**)

If there is a ... **3rd argument**, then it is moved to **%rdx** (or **%edx**, or **%dx** or **%dl**)

If there is a ... **4th argument**, then it is moved to **%rcx** (or **%ecx**, or **%cx** or **%cl**)

If there is a ... **5th argument**, then it is moved to **%r8** (or **%r8d**, or **%r8w** or **%r8b**)

If there is a ... **6th argument**, then it is moved to **%r9** (or **%r9d**, or **%r9w** or **%r9b**)

➤ **Callee:**

- Before returning to **caller**, **callee** must ensure that the **returned value** is loaded into its specific register, i.e., **%rax** (or **%eax**, or **%ax** or **%al**)

Who must ensure that ...?

Answer:

1. **gcc** -> when we compile our C program
2. **us** (software developers) -> when we write our own **x86-64** assembly programs

Passing data mechanism – Example of passing arguments in registers and returning return value

```
long plus(long x, long y) {
    return x + y;
}

void sum_store(long x, long y, long *dest)
{
    long sum = plus(x, y);
    *dest = sum;
}

int main(int argc, char *argv[]) {
    if (argc == 3) {
        long x = atoi(argv[1]);
        long y = atoi(argv[2]);
        long result;
        sum_store(x, y, &result);
        printf("%ld + %ld --> %ld\n", x, y, result);
    }
    else printf("2 numbers required!\n");
    return 0;
}
```

./ss 5 6

```
.globl sum_store
sum_store:
    endbr64
    pushq    %rbx
    movq     %rdx, %rbx
    call     plus
    movq     %rax, (%rbx)
    popq     %rbx
    ret
```

```
.globl main
main:
    endbr64
    pushq    %rbp
    pushq    %rbx
    subq     $24, %rsp
    movq     %fs:40, %rax
    movq     %rax, 8(%rsp)
    xorl     %eax, %eax
    cmpl     $3, %edi
    je       .L6
    leaq     .LC1(%rip), %rdi
    call     puts@PLT
.L3:
    movq     8(%rsp), %rax
    xorq     %fs:40, %rax
    jne      .L7
    movl     $0, %eax
    addq     $24, %rsp
    popq     %rbx
    popq     %rbp
    ret
```

```
.globl plus
plus:
    endbr64
    leaq     (%rdi,%rsi), %rax
    ret
```

```
.L6:
    movq     %rsi, %rbx
    movq     8(%rsi), %rdi
    movl     $10, %edx
    movl     $0, %esi
    call     strtol@PLT
    movslq   %eax, %rbp
    movq     16(%rbx), %rdi
    movl     $10, %edx
    movl     $0, %esi
    call     strtol@PLT
    movslq   %eax, %rbx
    movq     %rsp, %rdx
    movq     %rbx, %rsi
    movq     %rbp, %rdi
    call     sum_store@PLT
    movq     (%rsp), %r8
    movq     %rbx, %rcx
    movq     %rbp, %rdx
    leaq     .LC0(%rip), %rsi
    movl     $1, %edi
    movl     $0, %eax
    call     __printf_chk@PLT
    jmp      .L3
.L7:
    call     __stack_chk_fail@PLT
```

Let's **hand trace** this and see what is happening!

```

.global main
main:
    endbr64
    pushq    %rbp
    pushq    %rbx
    subq     $24, %rsp
    movq     %fs:40, %rax
    movq     %rax, 8(%rsp)
    xorl     %eax, %eax
    cmpl     $3, %edi
    je       .L6
    leaq     .LC1(%rip), %rdi
    call     puts@PLT
.L3:
    movq     8(%rsp), %rax
    xorq     %fs:40, %rax
    jne      .L7
    movl     $0, %eax
    addq     $24, %rsp
    popq     %rbx
    popq     %rbp
    ret

```

```

        .globl  plus
plus:
        endbr64
        leaq    (%rdi,%rsi), %rax
        ret

```

```
.L6:
    movq    %rsi, %rbx
    movq    8(%rsi), %rdi
    movl    $10, %edx
    movl    $0, %esi
    call    strtol@PLT
    movslq  %eax, %rbp
    movq    16(%rbx), %rdi
    movl    $10, %edx
    movl    $0, %esi
    call    strtol@PLT
    movslq  %eax, %rbx
    movq    %rsp, %rdx
    movq    %rbx, %rsi
    movq    %rbp, %rdi
    call    sum_store@PLT
    movq    (%rsp), %r8
    movq    %rbx, %rcx
    movq    %rbp, %rdx
    leaq    .LC0(%rip), %rsi
    movl    $1, %edi
    movl    $0, %eax
    call    __printf_chk@PLT
    jmp     .L3

.L7:
    call    stack_chk_fail@PLT
```

Register Table :

[illegible]

```

    .globl  sum_store
sum_store:
    endbr64
    pushq   %rbx
    movq    %rdx, %rbx
    call    plus
    movq    %rax, (%rbx)
    popq    %rbx
    ret

```

Stack - M[]

[illegible]